DEVELOPER_HANDBOOK

Audience: Engineers joining the SecurityBlogs project. **Goal:** Get productive within one day. Find any answer without asking.

---

1. First day onboarding

Required accounts

- GitHub access to `Jonaid880/Security-Blogs`
- Airtable access to `Securityblogs.com.au.` base
- Hostinger SSH key on the VPS (deploy user)
- Sentry seat (Phase D+)
- Plausible viewer (Phase D+)

Required software

Tool	Version	Install
Node.js	22 LTS	`winget install OpenJS.NodeJS.LTS`
pnpm	9 LTS	`npm i -g pnpm`
Docker Desktop	latest	https://docs.docker.com/desktop/
Python	3.12	For PDF doc generation
git	2.40+	(already on Windows)

Clone + run

```
git clone https://github.com/Jonaid880/Security-Blogs.git
cd Security-Blogs
git checkout phase-c-frontend-rewire

# Marketing site
pnpm install
cp .env.example .env.local            # then fill in keys

# CMS
cd cms
pnpm install
cp .env.example .env                  # then fill in keys
cd ..

# Stack
docker compose up -d                  # Postgres + MailHog

# DB init
cd cms
pnpm payload migrate
pnpm seed:all                         # populates Services, Pages, etc.

# Run
pnpm dev                              # CMS at http://localhost:3001/admin
# In second terminal:
cd ..
pnpm dev                              # Marketing at http://localhost:3000
```

2. Repository layout (quick reference)

Path	Contents
`app/`	Next.js App Router pages (marketing site)
`components/`	React components — `layout/`, `ui/`, `modules/`, etc.
`lib/cms.ts`	Typed Payload REST client — every page goes through here
`lib/cmsTypes.ts`	Read-shape TypeScript types
`lib/submitForm.ts`	Form helper — posts to `/api/leads`
`middleware.ts`	Edge middleware — CMS-driven redirects
`cms/`	Payload CMS app (separate Next.js install)
`cms/src/collections/*.ts`	Collection schemas
`cms/src/globals/Settings.ts`	Settings global schema
`cms/src/access/*.ts`	Role-check helpers
`cms/src/seed/*.ts`	Idempotent seed scripts
`cms/src/email/transport.ts`	nodemailer adapter
`docs/`	Phase-based engineering docs
`Documentation/Markdown/*.md`	Implementation docs (this folder)
`Documentation/PDFs/*.pdf`	Generated PDFs
`docker-compose.yml`	Postgres + MailHog for dev
`.github/workflows/deploy.yml`	**BROKEN** — was for static GitHub Pages deploy; rewrite in Phase D

3. Code conventions

TypeScript

- **Strict mode on.** All files compile under `tsc --noEmit`.
- Prefer `type` over `interface` for unions, prefer `interface` for object shapes you'll extend.
- No `any` in production code. Use `unknown` and narrow.

Naming

- Files: kebab-case for routes (`book-strategy-call/`), PascalCase for React components (`HeroBlock.tsx`).
- React components: PascalCase, one component per file.
- Server functions: camelCase, named after the resource (`getPage(slug)`).
- Env vars: SCREAMING_SNAKE; `NEXT_PUBLIC_*` for browser-safe.

Comments

- Default to no comments. Identifiers should self-explain.
- Add a comment only for non-obvious WHY: a hidden invariant, a workaround, a surprising behaviour.

- Never comment WHAT the code does.
- Multi-paragraph docstrings forbidden. One short line max.

Imports

- Absolute `@/lib/cms` over relative `../../lib/cms`. Configured in `tsconfig.json`.
- Group: external, then `@/...`, then relative. Blank line between groups.

Git

- Branch per phase or feature. Never push to `main` directly.
- Commit messages: imperative mood. First line ≤72 chars. Body explains why.
- Sign-off Phase A / B / C.1 in `docs/05-phases.md` after local validation.
- Never merge a branch with failing TypeScript or lint.

Pull requests

- Title: short imperative.
- Body: Summary + Test Plan + Rollback Plan.
- Link the Airtable per-page row(s) you touched.

4. Common workflows

Add a new CMS collection

1. Create `cms/src/collections/<Name>.ts` (use Posts.ts as template).
2. Register in `cms/payload.config.ts` collections array (sidebar order matters).
3. `cd cms && pnpm payload migrate` (auto-creates table in dev).
4. Optional: write `cms/src/seed/<name>.ts` and add to `package.json` scripts.
5. Update `lib/cmsTypes.ts` with read-shape TS type.
6. Add typed getter to `lib/cms.ts`.
7. Update `AIRTABLE_ARCHITECTURE.md` table inventory if relevant.
8. Add a per-page doc in `Documentation/Markdown/` if the new collection backs a page.

Add a new block type to Pages

1. Add to `cms/src/collections/Pages.ts` `modules` blocks array.
2. Migrate.
3. Create `components/modules/blocks/<Name>Block.tsx`.
4. Add dispatch case to `components/modules/ModuleRenderer.tsx`.
5. Update `lib/cmsTypes.ts` `CmsBlock` union if you want type-safe rendering.
6. Add a row to the relevant per-page table in Airtable describing the new block.

Add a new form

1. Create the form component (use `components/ui/ContactForm.tsx` as template).
2. Call `submitForm({ formData, subject, source: 'your-form-id' })` on submit.
3. Optionally extend Leads `SOURCE_OPTIONS` in `cms/src/collections/Leads.ts` to include your `source` value.

4. Wire Turnstile by rendering `<Turnstile />` inside the form (Phase C.2 will add to existing 5 forms too).
5. Test: submit form, verify Lead created in `/admin/collections/leads`.

Add a new page

1. Decide: hardcoded `app/<route>/page.tsx` OR CMS-driven (create a `Page` record in admin).
2. If CMS-driven: just add the slug + modules in admin; the dynamic route resolves it.
3. If hardcoded: write the `page.tsx` file. Plan to migrate to CMS in next sprint.
4. Add a row to `SB · 00 · Pages Index` in Airtable.
5. Create per-page table in Airtable (use template).
6. Write per-page doc in `Documentation/Markdown/`.

5. How to debug

"My page returns 404 from the CMS but exists"

- Status filter — `getPage()` requires `status='published'`. Drafts hidden from public.
- Slug mismatch — check the exact `slug` value in admin.
- Cache stale — try `getPage(slug, { revalidate: 0 })` once.

"ECONNREFUSED on lib/cms.ts fetch"

- CMS not running. Start it: `cd cms && pnpm dev`.
- CMS_URL env wrong. Default is `http://localhost:3001`. Check `.env.local`.

"Pages render fine but forms don't submit"

- Open browser devtools Network tab. Find the `/api/leads` POST.
- 400 = validation failed (response body has reason).
- 429 = rate limited (wait 15 min).
- 502 = CMS rejected the lead. Check CMS logs.

"Middleware not redirecting"

- Confirm middleware.ts is at repo root, not in a subfolder.
- Check matcher excludes the path you're testing (api/, _next/, admin/, files with `.`).
- Wait up to 60s for cache to refresh, or restart dev server.

"TypeScript errors in lib/cmsTypes.ts after CMS schema change"

- Run `cd cms && pnpm generate:types` to regenerate `cms/src/payload-types.ts`.
- Manually update `lib/cmsTypes.ts` to mirror the change (intentional — keeps read shape decoupled).

"Sharp install fails on Linux during deploy"

- Use Sharp's prebuilt Linux x64 binary. Ensure `glibc` >= 2.31 on Ubuntu.

6. Build and verify locally

```
# Type check both apps
pnpm tsc --noEmit
cd cms && pnpm tsc --noEmit
```

```
# Lint
pnpm lint
cd cms && pnpm lint

# Build (catches build-time errors)
pnpm build
cd cms && pnpm build

# Run smoke
pnpm start                # marketing site at :3000
cd cms && pnpm start      # CMS at :3001
```

7. Documentation workflow

Generate / regenerate PDFs

```
cd Documentation
python convert_md_to_pdf.py
```

Reads every `.md` in `Markdown/`, writes matching `.pdf` to `PDFs/`. Idempotent.

Add a new doc

1. Write the markdown in `Documentation/Markdown/<NAME>.md`.
2. Run the converter.
3. Add a row to `Documentation/DOCUMENTATION_INDEX.csv` (or sync via Airtable webhook).
4. Commit both `.md` and `.pdf`.

Style conventions for docs

- H1 = doc title (one per file).
- H2 = major sections.
- H3 = sub-sections.
- Tables for any structured comparison.
- Code blocks fenced with the language (`` `bash ``, `` `ts ``).
- Cross-reference other docs by filename, not full path.

8. Auth roles (cheat sheet)

Role	Can do	Cannot do
`super_admin`	Edit Settings global, change other users' roles, delete any record	Bypass deletion of self while logged in
`admin`	CRUD on content collections, manage leads, see all data	Edit Settings, change Users.role
`editor`	Create + edit drafts, publish own posts	Delete others' posts, edit users, edit Settings

Implementation: `cms/src/access/roles.ts` + per-collection `access` policies.

9. Environment variables (full inventory)

Marketing site `.env.local`

Var	Purpose	Public?
NEXT_PUBLIC_SITE_URL	Canonical URL (sitemap, og)	Yes
CMS_URL	Where to fetch from (server-only)	No
PAYLOAD_API_KEY	Auth for server-to-server CMS calls	No
NEXT_PUBLIC_TURNSTILE_SITE_KEY	Cloudflare widget site key	Yes
TURNSTILE_SECRET_KEY	Server-side verify	No
NEXT_PUBLIC_MAPBOX_TOKEN	Map widgets	Yes (referrer-restricted)
NEXT_PUBLIC_GMAPS_KEY	Google Maps	Yes (referrer-restricted)
WEBHOOK_SECRET	Inbound webhooks (future)	No

CMS `cms/.env`

Var	Purpose
NODE_ENV	`development` / `production`
PORT	Default 3001
NEXT_PUBLIC_SERVER_URL	Externally visible CMS URL
PAYLOAD_SECRET	48-byte hex JWT signer
DATABASE_URI	Postgres connection string
SMTP_HOST, SMTP_PORT, SMTP_SECURE, SMTP_USER, SMTP_PASSWORD	Hostinger SMTP
EMAIL_FROM_NAME, EMAIL_FROM_ADDRESS	Sender identity
SEED_ADMIN_EMAIL, SEED_ADMIN_NAME, SEED_ADMIN_PASSWORD	First admin (delete after first login)
MEDIA_LOCAL_PATH	Where uploads land
NEXT_PUBLIC_MEDIA_BASE_URL	Public base URL for media
TURNSTILE_SITE_KEY, TURNSTILE_SECRET_KEY	Optional CMS-side Turnstile use
PLAUSIBLE_DASHBOARD_URL	Optional

10. Where to find specific things

Looking for	Path
Why a page renders the way it does	`Documentation/Markdown/<page>.md`
How a lead becomes a record	`app/api/leads/route.ts` + `cms/src/collections/Leads.ts`
What endpoint serves a sitemap	`app/sitemap.ts`
What blocks are available on Pages	`cms/src/collections/Pages.ts` top of file

Looking for	Path
How redirects work	`middleware.ts` + `cms/src/collections/Redirects.ts`
How Lexical body is rendered to HTML	`components/modules/LexicalRenderer.tsx`
Who can edit what	`cms/src/access/*.ts` + per-collection `access` blocks
What lives in Settings	`cms/src/globals/Settings.ts`
Phased plan + sign-offs	`docs/05-phases.md`
Deployment steps	`Documentation/Markdown/DEPLOYMENT_GUIDE.md`
DB schema	`Documentation/Markdown/DATABASE_ARCHITECTURE.md`
Airtable role	`Documentation/Markdown/AIRTABLE_ARCHITECTURE.md`
Overall topology	`Documentation/Markdown/SYSTEM_ARCHITECTURE.md`
Current state by component	`BACKEND_AUDIT_REPORT.md`
Phased hour estimates	`IMPLEMENTATION_ROADMAP.md`

11. Things that are EASY to break (gotchas)

Foot-gun	Avoidance
Adding `output: 'export'` back to `next.config.mjs`	Will silently break ISR. Don't.
Importing from `cms/` in marketing site code	Decouple — use `lib/cms.ts` as the ONLY bridge
Hardcoded secret in `lib/site.ts` or anywhere committed	Move to `.env.local`. `git secrets --scan` in pre-commit.
`revalidate: 0` on a hot page	Negates ISR; every request hits CMS. Use sparingly (only for preview).
Deleting a service folder before adding the redirect	Old URLs 404. Always: add Redirect, then `rm -rf`
Changing a collection slug after launch	Breaks every URL referencing it. Treat as constant.
Pushing `cms/.env` to git	Treat as P0 incident. Rotate every secret.

12. Performance budgets

Metric	Target
Largest Contentful Paint (LCP)	< 2.5s on desktop, < 4s on 4G mobile
Time to Interactive (TTI)	< 3.5s
Total Blocking Time (TBT)	< 200ms
Cumulative Layout Shift (CLS)	< 0.1
First Input Delay (FID)	< 100ms

Metric	Target
Lighthouse Performance score	> 85 on landing pages
ISR cache hit ratio	> 95% in production
Lead submission round-trip	< 1500ms
CMS admin TTI	< 4s (acceptable; not visitor-facing)

13. Glossary

Term	Meaning
ISR	Incremental Static Regeneration (Next.js cache mode)
MCP	Model Context Protocol (external tool integrations like Ahrefs, Supermetrics)
Lexical	Meta's rich-text editor framework; stores content as JSON state
PAT	Personal Access Token (Airtable auth)
Phase A/B/C/D/E	Project milestones — see `docs/05-phases.md`
AI Visibility	Optimizing content to be cited by ChatGPT / Perplexity / Gemini etc.
AEO / AIO / GEO	Answer / AI / Generative Engine Optimisation — service-line names

---

End of DEVELOPER_HANDBOOK.md